

WEEK-3 – Business Analytics

Task Name:

Multi-Echelon Supply Chain Network Design using Mixed-Integer Linear Programming (MILP)

1. Introduction

A multi-echelon supply chain connects production facilities, distribution centers, and customer regions. When several locations are involved, selecting suitable distribution routes becomes a complex decision problem.

This project applies **Mixed-Integer Linear Programming (MILP)** to identify a low-cost supply chain network while satisfying production limits, distribution-center capacities, customer requirements, and facility-opening decisions.

The supply chain structure is:

Factories → Distribution Centers → Customer Zones

2. Objective

The main objective of this project is to **minimize the overall logistics expenditure**.

The model considers:

- Factory-to-distribution-center transportation cost
- Distribution-center-to-customer transportation cost
- Fixed operating cost of distribution centers
- Factory production capacity
- Distribution-center capacity
- Customer demand
- Minimum service-level requirements

The model helps the company select an economical and reliable supply chain configuration.

3. Supply Chain Network

For the revised example, the network contains:

Factories

- Factory 1 (F1)
- Factory 2 (F2)

- Factory 3 (F3)
- Factory 4 (F4)

Distribution Centers

- Distribution Center 1 (D1)
- Distribution Center 2 (D2)
- Distribution Center 3 (D3)

Customer Zones

- Customer 1 (C1)
- Customer 2 (C2)
- Customer 3 (C3)
- Customer 4 (C4)

Products move through:

Factories → Distribution Centers → Customers

4. Decision Variables

The model uses three major decision variables.

Factory to Distribution Center

x_{ij} represents the quantity of products transported from factory i to distribution center j .

Distribution Center to Customer

y_{jk} represents the quantity of products transported from distribution center j to customer k .

Distribution Center Opening

z_j is a binary variable.

- $z_j = 1 \rightarrow$ Distribution center is open
- $z_j = 0 \rightarrow$ Distribution center is closed

The binary variable makes the model a **Mixed-Integer Linear Programming** problem.

5. Input Data

Factory Capacity

Factory	Maximum Capacity
F1	450 units
F2	650 units
F3	550 units
F4	500 units

Distribution Center Capacity

Distribution Center	Capacity	Fixed Cost
D1	700 units	₹4,500
D2	850 units	₹5,500
D3	650 units	₹4,000

Customer Demand

Customer	Demand
C1	350 units
C2	450 units
C3	500 units
C4	400 units

Total customer demand:

$$350 + 450 + 500 + 400 = 1,700 \text{ units}$$

Total factory capacity:

$$450 + 650 + 550 + 500 = 2,150 \text{ units}$$

Therefore, the total factory capacity is sufficient to meet customer demand.

6. Transportation Costs

Factory to Distribution Center

	D1	D2	D3
F1	₹9	₹14	₹12
F2	₹11	₹8	₹10
F3	₹7	₹10	₹13
F4	₹12	₹9	₹6

Distribution Center to Customer

	C1	C2	C3	C4
D1	₹6	₹8	₹9	₹11
D2	₹9	₹5	₹7	₹8
D3	₹10	₹9	₹6	₹5

These values represent the cost of transporting **one unit** of product.

7. Objective Function

The model minimizes three major costs:

1. Factory-to-distribution-center transportation cost
2. Distribution-center-to-customer transportation cost
3. Fixed distribution-center operating cost

The objective function is:

$$\text{Minimize } Z = \sum_i \sum_j c_{ij} x_{ij} + \sum_j \sum_k c_{jk} y_{jk} + \sum_j f_j z_j$$

In simple terms, the model finds the **least-cost method of moving products from factories to customers**.

8. Constraints

Constraint 1 – Factory Capacity

A factory cannot transport more products than its production capacity.

$$\sum_j x_{ij} \leq S_i$$

Constraint 2 – Distribution Center Flow Balance

The quantity entering a distribution center must equal the quantity leaving it.

$$\sum_i x_{ij} = \sum_k y_{jk}$$

This prevents products from being lost or artificially created at a distribution center.

Constraint 3 – Distribution Center Capacity

A distribution center cannot process more products than its capacity.

$$\sum_k y_{jk} \leq H_j z_j$$

If the distribution center is closed, $z_j = 0$, so no products can pass through it.

Constraint 4 – Customer Demand

Each customer must receive the required demand.

$$\sum_j y_{jk} = D_k$$

Constraint 5 – Service Level

At least 90% of customer demand must be supported.

$$\sum_j y_{jk} \geq 0.90 D_k$$

Constraint 6 – Distribution Center Decision

$$z_j \in \{0,1\}$$

A distribution center can either be open or closed.

Constraint 7 – Non-Negativity

Transportation quantities cannot be negative.

$$x_{ij}, y_{jk} \geq 0$$

9. Python Implementation

```
import pulp
```

```
factories = ["F1", "F2", "F3", "F4"]
```

```
centers = ["D1", "D2", "D3"]
```

```
customers = ["C1", "C2", "C3", "C4"]
```

```
capacity = {  
    "F1": 450,  
    "F2": 650,  
    "F3": 550,  
    "F4": 500  
}
```

```
dc_capacity = {  
    "D1": 700,  
    "D2": 850,  
    "D3": 650  
}
```

```
fixed_cost = {  
    "D1": 4500,  
    "D2": 5500,  
    "D3": 4000  
}
```

```
demand = {  
    "C1": 350,  
    "C2": 450,  
    "C3": 500,  
    "C4": 400  
}
```

```
cost_fd = {  
    ("F1","D1"): 9, ("F1","D2"): 14, ("F1","D3"): 12,  
    ("F2","D1"): 11, ("F2","D2"): 8, ("F2","D3"): 10,  
    ("F3","D1"): 7, ("F3","D2"): 10, ("F3","D3"): 13,  
    ("F4","D1"): 12, ("F4","D2"): 9, ("F4","D3"): 6  
}
```

```
cost_dc = {
```

```

("D1","C1"): 6, ("D1","C2"): 8,
("D1","C3"): 9, ("D1","C4"): 11,
("D2","C1"): 9, ("D2","C2"): 5,
("D2","C3"): 7, ("D2","C4"): 8,
("D3","C1"): 10, ("D3","C2"): 9,
("D3","C3"): 6, ("D3","C4"): 5
}

```

```

model = pulp.LpProblem(
    "Multi_Echelon_Network",
    pulp.LpMinimize
)

```

```

x = pulp.LpVariable.dicts(
    "Factory_DC",
    [(i,j) for i in factories for j in centers],
    lowBound=0
)

```

```

y = pulp.LpVariable.dicts(
    "DC_Customer",
    [(j,k) for j in centers for k in customers],
    lowBound=0
)

```

```

z = pulp.LpVariable.dicts(
    "DC_Open",
    centers,
    cat="Binary"
)

```

```

model += (
    pulp.lpSum(
        cost_fd[i,j] * x[i,j]
        for i in factories
        for j in centers
    )
)

```

```

    )

    +

    pulp.lpSum(

        cost_dc[j,k] * y[j,k]

        for j in centers

        for k in customers

    )

    +

    pulp.lpSum(

        fixed_cost[j] * z[j]

        for j in centers

    )

)

for i in factories:

    model += pulp.lpSum(

        x[i,j] for j in centers

    ) <= capacity[i]

for j in centers:

    model += pulp.lpSum(

        x[i,j] for i in factories

    ) == pulp.lpSum(

        y[j,k] for k in customers

    )

    model += pulp.lpSum(

        y[j,k] for k in customers

    ) <= dc_capacity[j] * z[j]

for k in customers:

    model += pulp.lpSum(

        y[j,k] for j in centers

    ) == demand[k]

model.solve(pulp.PULP_CBC_CMD(msg=False))

```



```
print("Status:", pulp.LpStatus[model.status])  
print("Minimum Total Cost:",  
      pulp.value(model.objective))
```

10. Expected Result

After executing the Python program, the optimization solver will provide:

- Optimization status
- Minimum total supply-chain cost
- Factory-to-distribution-center shipment quantities
- Distribution-center-to-customer shipment quantities
- Distribution-center opening decisions

Example Output Format

Status: Optimal

Minimum Total Cost: XXXXX

Factory to Distribution Center:

F1 -> D1 : ...

F2 -> D2 : ...

F3 -> D1 : ...

F4 -> D3 : ...

Distribution Center to Customer:

D1 -> C1 : ...

D2 -> C2 : ...

D3 -> C4 : ...

Distribution Center Decisions:

D1 Open: 1

D2 Open: 1

D3 Open: 1

The exact optimal values are determined automatically by the MILP solver.

11. Business Interpretation

The optimized solution helps management make better logistics decisions.

It can identify:

- Which distribution centers should operate
- Which factories should supply each distribution center
- How many units should be transported
- Whether distribution-center capacity is sufficient
- Whether customer requirements can be satisfied
- Where transportation costs can be reduced
- How resources can be allocated efficiently

This allows the organization to make **data-driven supply chain decisions** rather than relying only on manual planning.

12. Conclusion

This project demonstrates the application of **Mixed-Integer Linear Programming** to a multi-echelon supply chain network.

The model combines factory capacity, distribution-center capacity, transportation costs, customer demand, service levels, and facility-opening decisions.

By optimizing these factors simultaneously, the company can develop a **cost-effective and efficient distribution network**.